

The Context Inversion Protocol: An Edge-First Profile for Persistent Human Context, with a Local Custody and Provenance Prototype

Robert E. Potter
Sovereign OS Project — Independent Research Initiative

Version 0.2.0
August 2026

Canonical HTML edition: <https://sovereignos.foundation/whitepaper/>
DOI: <https://doi.org/10.5281/zenodo.22313971>

Abstract

This paper names the **Context Inversion Protocol (CIP)** as Robert Potter’s edge-first implementation profile for persistent human context. CIP would assign authority to the local edge and treat external models, agents, and tools as replaceable utilities behind a policy boundary. It is layered over the Human Context Protocol and the Model Context Protocol: it uses HCP’s user-control semantics and MCP transport, together with the Global Memory Workshop’s five-component runtime sketch, without replacing those specifications or taking HCP’s preference store as the native object [1, 2, 3, 4]. HCP specifies a user-owned preference layer that authorized models may ingest [1]. CIP specifies mediated access to a provenance-bearing custody plane in which a captured source, a machine observation, and an operator-accepted claim remain distinct.

Version 0.2.0 reports an early Sovereign OS (SOS) prototype that emerged after the first edition. As of August 2026 that prototype is published as the Sovereign OS Control Plane 1.6.0, an MIT-licensed local control plane for source custody, deterministic evidence, reviewed knowledge, typed relations, and traceable retrieval [5]. It is offered as an author-reported existence claim: durable personal context can be kept in ordinary, inspectable local artifacts without becoming a hidden personality profile. The prototype is not presented as a completed CIP or HCP implementation, it is not an “HCP-MCP stack,” it is not a community-maintained production product, and it is not an independently verified result.

The distinction matters. A conventional AI memory or character profile may be visible as a short list of facts while the system that extracts, categorizes, ranks, and injects those facts remains vendor-controlled. SOS does not claim to automate self-knowledge or certify the quality of an operator’s judgment. Its present contribution is to make the construction of durable personal context visible, revisable, and evidence-bearing. The full CIP runtime remains proposed: it would add a policy and authorization service, a context router, mediated interoperability, and audit of delegated execution in front of the local custody plane described here.

Contents

1	Introduction	2
1.1	A Recurring Cycle: Centralization, Reclamation, and the Stakes of Memory	2
1.2	Positioning Statement	2

2	The Human Context Protocol Research Program	3
2.1	HCP: The Original Position Paper	3
2.2	The Expanded Working Paper: Actions, Audit, and an Expanded Author List	4
2.3	The Global Memory Workshop: A Five-Component Reference Architecture	5
2.4	What the HCP Research Program Leaves Open	5
3	Memory Architectures for Personal AI	6
3.1	Why Statelessness Forces an External Memory Architecture	6
3.2	Architectural Families	6
4	Sovereignty and User-Controlled Memory in Practice	8
4.1	Personal Data Stores: Solid and Inrupt	8
4.2	Local-First and On-Device Memory	8
4.3	How Commercial Products Expose or Withhold User Control	9
5	Delegated Reasoning and Agentic Orchestration	9
5.1	Tool Use and the Model Context Protocol	9
5.2	Agent-to-Agent Delegation	10
5.3	Task Decomposition and Human-in-the-Loop Trust Boundaries	10
6	The Context Inversion Protocol and the SOS Custody Plane	10
6.1	Scope of the Contribution	10
6.2	Prototype Description: An Evidence-Oriented Local Control Plane	11
6.3	Open-Source: A Licensed Control Plane, Not an HCP Implementation	12
6.4	Local Memory Graphing: A Custody Graph, Not Yet a Gated Preference Graph . .	12
6.5	Application and Task Assistance: Proposed CIP Work, Not Present SOS Behavior .	13
6.6	Proposed CIP Runtime Data Flow	14
7	Privacy, Security, and Governance	14
7.1	Regulatory Implications: GDPR and the Right to Erasure	14
7.2	Threat Model: Memory Poisoning	15
7.3	Threat Model: MCP Tool Poisoning	15
8	Comparative Positioning	16
9	Challenges and Future Directions	17
10	Conclusion	17

1 Introduction

1.1 A Recurring Cycle: Centralization, Reclamation, and the Stakes of Memory

The history of personal computing is a repeating argument about where state lives. Mainframe timesharing centralized computation inside institutions and rented cycles back to users by the minute. The personal computer revolution of the late 1970s and 1980s inverted that default: computation, storage, and control moved onto a machine an individual owned outright. Cloud computing and software-as-a-service reversed the inversion a second time, moving files, identity, and increasingly cognition itself back into infrastructure controlled by a small number of vendors. Tim Berners-Lee’s

Solid project is an explicit attempt to run the reclamation half of that cycle again for personal data specifically, built on the premise that personal information should be “centred around people, instead of around apps” [6]. Solid’s Pods are personal online data stores where users “store information once and decide who can access what” [7], governed by Inrupt’s “fine-grained access controls” set by the owner rather than the application requesting access [8]. Security technologist Bruce Schneier summarized the model plainly: “Your data lives in a pod that is controlled by you ...You authorize granular access to that pod to whoever you want” [9]. Real deployments already exist at civic scale: Flanders’ data-utility company Athumi piloted a Solid-based recruitment application built with staffing firm Randstad and planned to scale it to 30,000 users during 2024, with Athumi’s chief executive describing the underlying shift as one where “the data subject becomes the data controller”; separately, five Belgian hospitals began storing hospital-visit records inside patients’ own Pods rather than a hospital-controlled database [6].

Personal AI memory is where this cycle reaches its highest stakes yet. A model that remembers a user’s health conditions, relationships, finances, and unfinished projects across sessions is not a convenience feature – it is a second copy of a person’s cognitive context, and whoever controls its storage, its access policy, and its export path controls a durable share of that person’s relationship with every AI system they will ever use. The Sovereign OS Project takes its name, and its founding argument, from this history: that the correct default for persistent human context is the personal-computer default, not the cloud-timesharing default, and that the recent flourishing of academic and institutional work formalizing related positions – discussed at length below – is evidence that the argument has moved from a philosophical stance to an engineering research program with named protocols, a growing set of prototypes, and an unresolved question about what the stored object actually is.

1.2 Positioning Statement

The objects of this paper are three nested claims, and they must not be collapsed.

1. **The Sovereign OS Project** is an independent research initiative. Its question is where authority over persistent human context should reside as AI systems become more capable.
2. **The Context Inversion Protocol** is the project’s hypothesis: that authority belongs at the edge; that models, agents, and tools are replaceable utilities; and that every external read, proposed write, and delegated action should cross a local policy boundary. CIP composes HCP’s user-control semantics with MCP’s tool transport and the Global Memory Workshop’s five-component architecture [1, 2, 3, 4]. It does not claim to originate those specifications.
3. **Sovereign OS (SOS)** is a published prototype of the first layer such a protocol would require: a local custody plane in which sources, machine observations, and accepted human knowledge remain distinct, inspectable, and revisable. Version 1.6.0 is the artifact described in this edition [5]. It is not CIP. It is not HCP. It does not presently expose MCP.

Those three objects remain distinct without HCP. SOS is the local custody plane: captured sources, deterministic evidence, and operator-accepted knowledge, kept as ordinary files. CIP would add a mediated runtime in front of that plane – a local gateway, purpose-limited grants, provenance-bearing bundles, and a ledger of disclosure – so replaceable models and tools never mount the store. HCP is not required for either layer to exist. It is the protocol-family vocabulary this paper uses when the remaining problem is scoped sharing with AI systems.

HCP’s December 2025 expanded working paper and a March 2026 multi-institution workshop report

on the same protocol family establish that mediated access, store operations, and audit are named parts of that protocol family rather than gaps left open by it. The expanded paper specifies Read, Write, Update, and Delete actions over the protocol’s data store, exposable as REST endpoints or Model Context Protocol (MCP) tools, and states that “transparent audit logs allow users to review exactly which agents accessed which data and when” (Section 2.2) [2]. The workshop report goes further, decomposing a user-governed context layer into five named components – Memory Store, Policy & Authorization Service, Context Router (Memory Manager), Audit & Provenance Subsystem, and Interoperability Interface – with the Interoperability Interface defined around add, search, update, and delete operations [3].

The claim this paper defends is therefore two-layered, and the second layer is not a restatement of the first. HCP’s research program has moved from position paper to architectural blueprint, while leaving open an independently auditable, local, and operational implementation of its combined components. That blueprint, however, still treats the stored object as a preference or personalization layer that authorized models ingest [1]. Rebuilding HCP at the edge – a local PostgreSQL of categorized facts, exposed over MCP, with better consent – would leave the silent-profile problem intact. The SOS prototype is offered as an author-reported local existence claim of a different kind: a custody and evidence plane can make persistent context inspectable before a model is asked to reason over it, and it can refuse to treat a personality profile as the native record. It does not yet provide the policy and authorization service, context router, interoperability interface, or audited delegated execution required for a CIP runtime. The prototype therefore narrows the research program rather than resolving it. It identifies a feasible substrate for sovereign context and makes the remaining enforcement problem explicit.

2 The Human Context Protocol Research Program

2.1 HCP: The Original Position Paper

The Human Context Protocol was introduced by Anand V. Shah and Michiel A. Bakker (MIT), Tobin South (MIT), Talfan Evans (Persona Labs), Hannah Rose Kirk (University of Oxford / UK AI Security Institute), Andrew Trask (OpenMined), and E. Glen Weyl (Microsoft Research), who argue that “robust AI personalization requires a Human Context Protocol (HCP): a user-owned, secure, and interoperable preference layer that grants individuals granular, revocable control over how their data steers AI systems” [1]. The paper specifies four design attributes any conforming system must satisfy: **Interoperable** (facilitated by “open, well-supported, and existing communication protocols”); **Sufficiently Representative** (a data model “at least as expressive as state-of-the-art techniques in preference capture”); **Scopable Sharing** (“fine-grained, revocable, and editable control over what preference information is shared and with whom ...a user should be able to share culinary preferences with a recipe generator without exposing mental health information”); and **Secure** (“preferences must be secured at rest and in transit, with strong authentication and authorization mechanisms”) [1].

The authors built and shipped a proof-of-concept: a web application in which users manage preferences and third-party access, backed by an integrated MCP server that enforces “user authentication and granular authorization for distinct preference categories,” with preferences “stored as categorized textual data in PostgreSQL” and a GPT-4o-based “HCP LLM” that “ingests only authorized preference categories” per query [1]. The reference implementation is publicly available [10]. That repository (github.com/tobinsouth/hcp-demo), maintained by co-author Tobin South as a Stanford Digital Economy Lab / Consumer Reports collaboration, is in practice a Next.js application that

stages the protocol’s story across three progressive scenes – Human Context, Grant of Authority, and Applications – rather than exposing a runnable backend a third party could deploy and audit end to end. As of August 2026 it carries no license file and no tagged release, a discrepancy with the original paper’s own text, which describes this same prototype as “open-source” [1, 10]. This is not a criticism of the demo’s communicative purpose, but it is the first of several concrete gaps between HCP’s stated intentions and its shipped artifacts. Critically, the original paper is explicit that the prototype is minimal and that specification, not implementation, is its contribution: “this paper does not prescribe a definitive implementation for HCP; any system fulfilling the aforementioned design attributes would be suitable” [1].

2.2 The Expanded Working Paper: Actions, Audit, and an Expanded Author List

The protocol’s expanded working paper, retitled “Robust AI Personalization Controls: The Human Context Protocol,” dated 18 December 2025 by Stanford Digital Economy Lab, and co-authored with Jiaxin Pei (Stanford), expands the specification in ways that materially change what a downstream implementer can claim as novel [2]. The authors do not number this document as a separate protocol version. Two changes matter most for this paper’s positioning. First, the four design attributes are renamed and one is substantively redefined: **Interoperability**, **Encapsulation**, **Control**, and **Security**. Encapsulation states that the data model “should leverage advances in preference representation (whether as text, graph-based knowledge structures, vector embeddings), provided those representations are portable” [2]. A local graph is therefore a permitted HCP representation, not a forbidden analogy. Encapsulation does not rank graphs above text or vectors, and a graph does not by itself satisfy Control or Security. Section 6.4 returns to this permission: the SOS prototype uses typed local records; it does not claim to have satisfied Encapsulation as a gated, portable preference interface.

Second, the expanded paper specifies four store actions – Read, Write, Update, and Delete – exposable as REST endpoints or as MCP tools, and states that “transparent audit logs allow users to review exactly which agents accessed which data and when” [2]. An audit layer is therefore a named, specified part of the protocol itself, not an unaddressed gap. What that paper does not establish is a *working, open-source, community-auditable implementation* of that audit layer; it remains a specification accompanied by a single reference prototype that is publicly visible but carries no open-source license, no tagged releases, and no independent verification in production use [10].

2.3 The Global Memory Workshop: A Five-Component Reference Architecture

The most consequential document for scoping CIP’s remaining runtime is not a paper by the original HCP authors at all, but the unsigned final report of the March 11–12, 2026 Global Memory Workshop, titled “The Future of Personal AI: Portable and Persistent Personal Memory through a Unified Human Context Protocol” [3]. The report lists the Gates Foundation, AWS, the Mozilla Foundation, Slalom, and Stanford HAI on its footer; its executive summary names the Gates Foundation, Mozilla Foundation, Stanford HAI, and AWS as co-organizers. Stanford Digital Economy Lab hosts the PDF. The title places the report in the HCP protocol family; the PDF itself does not print a personal author byline.

The workshop report decomposes a user-governed context layer into five named architectural components [3]:

1. **Memory Store** – a durable store of human-readable entries with machine-usable metadata, and an optional vector index.
2. **Policy & Authorization Service** – token validation and least-privilege, category-scoped grants with revocation.
3. **Context Router (Memory Manager)** – selects what to disclose given the request, permitted scopes, and minimization rules.
4. **Audit & Provenance Subsystem** – an append-only log of memory accesses and mutations.
5. **Interoperability Interface** – constrained add, search, update, and delete operations mediated by the policy layer.

Orchestration, audit, and interoperable execution – capabilities that a reading of the original 2025 HCP position paper alone might suggest are unaddressed – each map onto a named component in this five-part architecture. The protocol ecosystem around HCP is, as specification, considerably more built out than the original position paper alone would indicate. CIP’s proposed mediation layer is that architecture placed in front of a local custody plane, with MCP as gateway transport rather than as SOS’s native API. The resemblance is intentional. CIP is an implementation profile for an existing stack, not a competing five-component diagram.

That said, the workshop report is equally explicit about what remains unresolved, and its own language is the strongest available textual anchor for the gap still open after the SOS prototype. A dedicated section on “On-Device and Small Model Constraints” calls for “distillation and fine-tuning toward memory tasks,” “hybrid pipelines combining small models with deterministic components,” and “tiered execution, where local models handle sensitive operations and only escalate to larger models under explicit permission” [3]. None of these are described as solved – they are described as requirements for future work. The original HCP preprint frames its own prototype as “an early step” rather than a production system [1]. The workshop report frames the core systemic risk as consolidation of the memory layer “inside a handful of frontier model providers” [3] – language that directly reinforces this paper’s sovereignty framing in Section 1.

2.4 What the HCP Research Program Leaves Open

Synthesizing the three publications, three concrete gaps remain open as of August 2026:

1. **No open-source, production-oriented implementation.** HCP’s own prototype is a single research demo built to validate the specification, not a community-maintained, auditable, redistributable codebase intended for other developers to run and extend in production.
2. **No working local memory graph.** The workshop’s own call for on-device, small-model, tiered execution is unaddressed by any of the three publications’ reference implementations, all of which store preferences in a centralized PostgreSQL instance rather than a local, portable graph structure.
3. **No demonstrated agentic delegation layer.** While the expanded HCP paper specifies Read/Write/Update/Delete actions and audit logging over the memory store itself, none of the three publications demonstrate a sub-agent or third-party tool actually executing a delegated task under the protocol’s consent and audit guarantees – the protocol governs access to context, not the downstream use of that context to act.

Version 0.2.0 separates a current prototype from the larger proposal. The SOS prototype is an author-reported local existence claim on the second gap: typed records, deterministic evidence, and retained sources, expressed as ordinary files rather than as a preference database. Version 1.6.0 supplies a public, MIT-licensed control plane [5]; it does not supply a production-oriented community project, a policy boundary, or delegated task execution. It also names a prior problem that HCP treats as given. HCP’s object is a preference layer that grants individuals control over how their data steers AI systems [1]. SOS’s object is the construction of durable context before any such layer is allowed to form. The proposed CIP runtime would add the missing mediation and execution layers in front of that custody plane; it would not replace the plane with a categorized profile.

3 Memory Architectures for Personal AI

3.1 Why Statelessness Forces an External Memory Architecture

Large language models are fundamentally stateless: each inference draws only on the tokens placed in a bounded context window, and nothing persists once that window closes. The MemGPT paper frames the problem directly, observing that LLMs “are constrained by limited context windows, hindering their utility in tasks like extended conversations and document analysis” [11]. The Mem0 authors make the same point about long-horizon interaction specifically: “fixed context windows pose fundamental challenges for maintaining consistency over prolonged multi-session dialogues” [12].

Simply enlarging the window does not resolve this. Anthropic’s engineering guidance describes “context rot,” whereby “as the number of tokens in the context window increases, the model’s ability to accurately recall information from that context decreases,” a property that “emerges across all models” because the transformer architecture’s pairwise attention computation is “stretched thin” at long context lengths [13]. Context must therefore be treated “as a finite resource with diminishing marginal returns,” the premise behind the discipline of context engineering: “the set of strategies for curating and maintaining the optimal set of tokens ...during LLM inference” [13].

3.2 Architectural Families

Retrieval-augmented generation (RAG) embeds text into vectors, stores them in a vector database, and retrieves the most semantically similar chunks at query time. It is simple and cheap to write, but stale and imprecise when embeddings blur distinct facts, and it cannot represent evolving relationships or temporal validity – a direct challenge to HCP’s Sufficiently Representative / Encapsulation attribute if used naively as a preference-capture mechanism.

Hierarchical, OS-style paging, introduced by MemGPT (now developed as Letta), draws “inspiration from hierarchical memory systems in traditional operating systems” and pages information between an in-context working set and an external store via model-issued function calls [11]. This is the architectural metaphor the Sovereign OS Project borrows for its “Operating System” framing: paging and custody, not a consumer operating system.

Memory streams with reflection, from the Stanford Generative Agents work, record “a comprehensive list of the agent’s experiences” in natural language and periodically synthesize higher-level insights through a reflection process that fires when accumulated importance scores cross a threshold [14]. Retrieval combines recency, importance, and semantic relevance as a weighted score [14].

Temporal knowledge graphs, exemplified by Zep, organize memory into episode, entity, and community subgraphs with a bi-temporal model tracking both when a fact was true in the world and when it was ingested, enabling “edge invalidation” – marking superseded facts outdated while preserving history – rather than deletion [15]. Zep’s production benchmark page reports 94.7% accuracy on LoCoMo and 90.2% on LongMemEval [16], though these figures are vendor self-reported; a March 2026 comparison published by Vectorize – itself a vendor of a competing memory product, Hindsight, which the same comparison recommends evaluating – reported Zep at only 63.8% and Mem0 at 49.0% on LongMemEval using GPT-4o, well below either vendor’s own claims [17]. Neither side of this comparison is a neutral source: the divergence between Zep’s and Mem0’s self-reported figures and a rival vendor’s unfavorable re-measurement of both is itself a standing caution against treating any single vendor-published memory benchmark as load-bearing evidence for an architecture decision.

Managed extraction pipelines, exemplified by Mem0 and its graph-augmented variant Mem0^g, “dynamically extract, consolidate, and retrieve salient information,” reporting “26% relative improvements in the LLM-as-a-Judge metric over OpenAI” on the LOCOMO benchmark, with the graph variant scoring roughly two points higher again [12] – a figure self-reported by Mem0’s own co-founders (the paper’s listed authors include Mem0’s CEO and CTO) rather than an independently verified result, the same caveat that applies to the Zep figures discussed above.

Approach	Strengths	Weaknesses / Risks
RAG / vector store	Simple, cheap writes, scalable	Staleness; retrieval imprecision; no relational or temporal modeling [12]
Hierarchical paging (MemGPT/Letta)	Extends effective context; self-managed	Added control-flow complexity [11]
Memory stream + reflection (Generative Agents)	Captures recency/importance/relevance; compresses via reflection	Hand-tuned weights; reflection cost [14]
Temporal knowledge graph (Zep)	Relational reasoning; fact invalidation; non-lossy history	Graph construction overhead; vendor benchmarks unverified independently [15, 17]
Managed extraction (Mem0/Mem0^g)	Strong benchmark accuracy; large token savings	Extraction-LLM dependence; sensitive-data leakage risk; founder-authored benchmark, unverified independently [12]
HCP preference store	Simple; MCP-native; scoped sharing; revocable	Narrow scope (preferences, not general graph memory); centralized PostgreSQL [1]

None of the general-purpose memory systems above were designed against HCP’s Scopable Sharing / Control attribute – fine-grained, revocable, per-recipient disclosure – and none, including HCP’s own reference prototype, implement the local graph representation the expanded paper’s Encapsulation attribute explicitly permits [2]. Those remain openings for a CIP runtime. They are not, taken together, a description of the SOS prototype. Extraction pipelines such as Mem0 are the nearest product analogue to the silent-profile pattern the prototype refuses: salient facts are model-extracted, consolidated, and later injected, with the construction of those facts remaining inside the memory system rather than as inspectable source, observation, and accepted claim.

4 Sovereignty and User-Controlled Memory in Practice

4.1 Personal Data Stores: Solid and Inrupt

The most developed standards effort for user-owned data remains the W3C Solid project. Inrupt’s implementation uses per-resource Access Control Policies and verifiable credentials, such that “if a resource has no Policies that apply to it, the resource is inaccessible” by default [8]. HCP’s own authors situate their work in this lineage explicitly, citing “earlier work in personal data stores” as the foundation they extend to “the scale and stakes of modern generative AI” [1]. Solid demonstrates that standards-based personal data sovereignty is deployable at civic scale, but its application-developer ecosystem remains comparatively small relative to the AI personalization space HCP targets.

4.2 Local-First and On-Device Memory

A parallel movement builds memory that never leaves the user’s hardware, and by 2026 it has moved well past forum speculation into shipped, benchmarked systems. Stanford’s Hazy Research and Scaling Intelligence Lab, together with Lambda Labs, released OpenJarvis in mid-2026: an Apache-2.0 on-device framework that runs inference, agent state, and interchangeable memory backends locally, with an adapter for external MCP servers as tools, landing within 3.2 percentage points of frontier cloud-model accuracy across eight benchmarks while cutting marginal cost roughly 800-fold and end-to-end latency roughly four-fold [18]. Apple Intelligence embodies the on-device model at consumer scale: its “cornerstone ...is on-device processing, so it is aware of your personal information without collecting your personal information,” and includes an “on-device semantic index that can organize personal information from across apps” [19], with a Private Cloud Compute tier for heavier workloads that processes “only the data that is relevant to the task” and is not retained [20]. Independent academic verification of such vendor claims remains rare and technically demanding: the first published reverse-engineering study of Private Cloud Compute had to build its own tooling just to query the system outside Apple’s intended interfaces, noting “there are no reproducible builds, and there are no symbols within those binaries,” and found the underlying proprietary model’s independently benchmarked accuracy comparable to, but below, Apple’s own published figures [21] – a reminder that sovereignty claims require independent verification rather than vendor assertion, a standard this paper applies to its own claims below. Neither OpenJarvis nor Apple Intelligence implements HCP’s specific consent, revocation, or scoped-sharing semantics: OpenJarvis is a general-purpose local-agent framework with no per-recipient access control over what a connected tool may read from memory, and Apple Intelligence exposes no interoperable protocol for third parties to request scoped access at all. The remaining gap is therefore not “local, on-device AI” – OpenJarvis already demonstrates that convincingly – but mediated access to a local custody plane that is not itself an ungated memory backend or a vendor semantic index. That is CIP’s remaining work. It is not a claim that SOS already supplies HCP’s consent-scoped interoperability layer.

4.3 How Commercial Products Expose or Withhold User Control

Commercial assistants have converged on a common structural pattern – “compressed long-term memory paired with raw working memory” [22] – while diverging sharply in transparency and portability.

Product	Storage	User Control	Portability
ChatGPT	Cloud	Manage, per-item delete, clear all [23]	Not documented in fetched sources
Claude	Cloud	Per-item + clear all, Temporary Chat [24]	Import tool for other chatbots’ exports [25]
Gemini	Cloud	Toggle, delete, Temporary Chats [26]	Not documented in fetched sources
Copilot / Recall	Local (encrypted)	Exclude apps/sites, delete ranges, uninstall [27]	Local DB on device
Perplexity	Cloud (encrypted)	Per-item + bulk clear; 30-day cleared log [28]	Not documented in fetched sources
Apple Intelligence	On-device + PCC	Per-app iCloud toggles; exportable PCC report [20]	Local device storage
HCP prototype	Cloud (PostgreSQL)	Web UI; per-category authorization [1]	Interoperable by design (MCP-native)

User control has improved across the industry – deletion, project scoping, and temporary modes are now common – but standards-based, full-export portability remains weak. Only Claude documents an import path for other vendors’ exports [25], and no fetched vendor source documents a full, standards-based export of its memory store. This paper takes HCP as the protocol-level anchor for scoped disclosure rather than as a competitor to differentiate against. Visibility of a fact list is not the same as inspectability of how those facts were constructed. The commercial pattern above is the problem statement for SOS’s custody plane; HCP’s scoped-sharing semantics are the problem statement for CIP’s still-unbuilt gate.

5 Delegated Reasoning and Agentic Orchestration

5.1 Tool Use and the Model Context Protocol

Anthropic’s Model Context Protocol (MCP), announced November 25, 2024, standardizes the boundary between a model and external tools as “a universal, open standard for connecting AI systems with data sources, replacing fragmented integrations with a single protocol” [4]. MCP servers expose Tools, Resources, and Prompts over JSON-RPC 2.0, and the specification treats host-level confirmation of tool use as a SHOULD-level recommendation rather than a mandatory guarantee, leaving the precise interaction model to the host implementation [29]. Secondary commentary sometimes states this more strongly – “every tool call goes through user approval” – which is an accurate description of intended host practice, not of a protocol MUST [30]. Adopted subsequently by OpenAI, which added MCP support across its Agents SDK and ChatGPT products after its CEO said “people love MCP and we are excited to add support across our products” [31], and by Google DeepMind, which added native MCP support to the Gemini API and SDK [32], MCP is, nearly two years after its release, “the closest thing the AI industry has to an actual standard” [30]. This is the same protocol family HCP’s own reference prototype uses as its interoperability transport [1]. The workshop report names an Interoperability Interface as a constrained add/search/update/delete tool surface; it does not name MCP as that interface’s required transport [3]. CIP would use MCP at its gateway: a local MCP and API surface through which replaceable models request scoped context without mounting the custody plane. SOS does not presently expose such a server. An operator may use a conversational agent against SOS’s local control plane; that arrangement is a trusted-authoring workflow, not CIP’s mediated runtime, and it is not an HCP-compliant MCP

implementation.

5.2 Agent-to-Agent Delegation

Google’s Agent2Agent (A2A) protocol, introduced April 2025 and later contributed to the Linux Foundation, standardizes how “different, independent AI agents communicate and collaborate with each other” [33]. A2A advertises agent capabilities via signed “Agent Cards” and, critically, “allow[s] agents to collaborate without needing to share internal memory, proprietary logic, or specific tool implementations” [34], preserving opacity between a delegating agent and the agents or tools it delegates to. A2A and MCP are explicitly complementary: A2A “complements Anthropic’s Model Context Protocol (MCP), which provides helpful tools and context to agents” [33]. HCP’s own paper gestures at A2A as a candidate integration path but, consistent with the gaps identified in Section 2.4, does not specify how a preference layer would mediate or audit cross-agent delegation in practice [1]. CIP would inherit that unsolved mediation problem; SOS does not claim to have solved it.

5.3 Task Decomposition and Human-in-the-Loop Trust Boundaries

The prevailing orchestration pattern dispatches specialized sub-agents that “handle focused tasks with clean context windows” and return “only a condensed, distilled summary of its work,” preserving a clear separation of concerns between exploration and synthesis [13]. Both major delegation protocols push consent decisions to a human: MCP’s specification states that when servers request completions via sampling, “there SHOULD always be a human in the loop with the ability to deny sampling requests,” and that applications “SHOULD” present confirmation prompts for tool calls, while explicitly noting that “the protocol itself does not mandate any specific user interaction model” [29, 30]. This human-in-the-loop guidance is the primary control preventing a delegated sub-agent or tool from acting beyond its authorized scope – and, as Section 7 shows, it is necessary but not sufficient on its own. The SOS prototype already treats human review as the only path from proposed synthesis to durable knowledge. That is an authoring-time gate, not a runtime authorization service for third-party tools.

6 The Context Inversion Protocol and the SOS Custody Plane

6.1 Scope of the Contribution

The Context Inversion Protocol is the implementation profile authored by Robert Potter for the topology developed in this paper. As specified here, CIP would assign authority over persistent human context to the local edge, require every external disclosure to pass through a consent-scoped policy boundary, and treat remote models, agents, and tools as replaceable compute utilities rather than context owners. CIP composes HCP’s user-control semantics with MCP’s tool transport and the Global Memory Workshop’s five-component architecture; it does not claim to originate those underlying specifications.

CIP is not a rebuilt Human Context Protocol. HCP answers how a user-owned preference layer can be shared with AI systems interoperably [1]. SOS answers how durable context can be constructed and kept inspectable under individual authority. CIP answers how that plane can later be disclosed to replaceable models and tools without becoming a silent profile a model is assumed to “know.” The workshop’s five components remain the right runtime sketch for that disclosure problem [3].

They are not, by themselves, a specification of the store those components would be allowed to touch.

Sections 2 through 5 establish that the HCP research program already specifies interoperability, representational encapsulation (including graph-based representations), scoped consent, and audit logging as protocol-level requirements, and that the broader field has separately developed strong memory-architecture and delegation-protocol building blocks. Version 0.1.0 proposed that Sovereign OS would instantiate the CIP profile and that stack end to end. Version 0.2.0 withdraws the completion claim. This edition reports the SOS control plane (version 1.6.0) as a published local custody and provenance prototype [5]. A CIP runtime would sit in front of a plane like SOS; it would not replace it, and SOS would not become CIP by renaming. The custody plane is now a permissively licensed public artifact; the CIP mediation layer remains a proposed runtime rather than shipped software. This paper publishes the protocol claim; it does not claim that the published control plane is that runtime.

6.2 Prototype Description: An Evidence-Oriented Local Control Plane

Sovereign OS is a transparent local control plane for durable human context. It does not create a hidden personality profile or delegate judgment to a model. It preserves source material, produces inspectable evidence, and lets the operator explicitly accept or revise the knowledge that becomes durable.

The mechanism is a three-tier arrangement, and that arrangement is the product distinction:

- **Tier 3** is captured reality: unmodified sources under local archive custody.
- **Tier 2** is deterministic observation: hash-bound extractions, indexes, and, where explicitly authorized, provenance-bound model artifacts that remain evidence rather than knowledge.
- **Tier 1** is accepted knowledge: operator-reviewed records with canonical identifiers and typed relations.

Capture, ingest, debrief, and retrieve is the operating loop that realizes that philosophy. It is not a second-brain workflow and not a claim that the loop itself is the contribution. The published control plane implements that loop as a local command-line interface together with conversational review protocols; it stores no provider key and does not run a hosted API [5].

Reviewed knowledge is Markdown with YAML identifiers and five typed predicates (`DERIVES_FROM`, `GOVERNS`, `IMPLEMENTS`, `EVIDENCES`, `TRANSMUTES`). Deterministic evidence is hash-bound companion files, including JSONL indexes keyed by SHA-256 of the source bytes. Sources remain archived originals. Domain charters declare an information-flow class (`private`, `restricted`, or `public`) that structural audit enforces among records; those classes are not HCP-style grants to external models. A pending debrief record is a control object, not knowledge; a model may assist with synthesis, but a durable Tier 1 record is created only after the operator approves the proposed text and its evidence route.

This architecture distinguishes three propositions that character-profile systems commonly conflate: a source exists, a process observed or extracted something from it, and an operator accepts a statement about it. The distinction does not make the operator infallible, and it is not a cryptographic proof that every sentence has been read. It makes the site of judgment explicit. Evidence may be traced through an indexed deterministic artifact to the retained source, using the most precise truthful locator that the medium supports: a source line or row where available, otherwise a transcript

segment and timestamp, page, image observation, or archive reference. The prototype’s audit validates structural relations, information-flow boundaries, and evidence custody; it does not yet mediate external model access at runtime.

The prototype’s non-goals are as load-bearing as its goals. SOS is not a second-brain application. It is not an autonomous life manager. It is not a universal personal profile. It is not an agent runtime with broad ambient access to the operator’s files. A conversational agent may be pointed at a living instance as a trusted author; durable writes still pass a review gate. That is not ambient access to the rest of the operator’s filesystem, and it is not CIP’s capability-gated disclosure.

Prototype element	Present function	CIP status
Tier 1 reviewed records	Local, operator-accepted knowledge in ordinary files	Candidate memory-store representation; not a policy-enforced shared store
Tier 2 deterministic evidence	Hash-bound extractions, indexes, and model-artifact contracts	Provenance substrate; not a disclosure ledger
Tier 3 retained sources	Recoverable original captures under local custody	Local ground source; access is not yet capability-gated
Graph, trace, and audit	Typed relations, retrieval, evidence-route inspection, structural validation	Partial audit/provenance foundation; no runtime access audit
Conversational debrief	Human review protocol for proposed synthesis	Proposal workflow; not an authorization service

6.3 Open-Source: A Licensed Control Plane, Not an HCP Implementation

HCP’s own authors invite independent implementations, having stated from the outset that the protocol does not prescribe a definitive implementation and that any system satisfying its design attributes is suitable (Section 2.1) [1]. Their own reference implementation, while publicly visible on GitHub, carries no open-source license file, no tagged releases, and no visible external contributor activity beyond its original authors [10], and none of the three publications surveyed in Section 2 describe a community governance model, a public issue tracker, or a versioned release process for their prototype. A protocol whose only reference implementation lacks a license, a release process, and community governance has not actually achieved the decentralization of control its specification calls for, no matter how carefully that specification is written.

The SOS control plane 1.6.0 is a public MIT-licensed artifact [5]. A third party can now read, run, and fork the custody-plane code. That is a narrower fact than HCP’s missing licensed implementation of HCP itself: SOS is not an HCP implementation, and it is author-maintained rather than community-governed. The public tree is a portable overlay; living-instance notes are not in that tree. Independent verification of behavior remains future work. A later CIP-conformant mediation layer would require a separate, equally inspectable release. Until that layer exists, claims about independent verifiability of CIP do not apply.

6.4 Local Memory Graphing: A Custody Graph, Not Yet a Gated Preference Graph

The expanded working paper’s Encapsulation attribute permits graph-based data representations, provided they remain portable (Section 2.2) [2]. None of the three publications’ reference implementations exercise the graph option; all store preferences in a centralized PostgreSQL instance. The workshop’s on-device and small-model constraints, discussed in Section 2.3, call for local, tiered

execution [3]. The SOS prototype occupies part of that opening: reviewed knowledge, evidence, and sources are local, portable, and related by typed edges. It does not occupy the rest. The prototype graph is not exposed through the workshop’s Interoperability Interface, is not gated by HCP Control, and is not an MCP memory server.

A concrete example that local graph-based memory already ships inside HCP’s own protocol family – rather than as an analogy borrowed from an unrelated domain – comes from the Model Context Protocol’s own reference server suite. The official `@modelcontextprotocol/server-memory` package implements “a basic implementation of persistent memory using a local knowledge graph” that “lets Claude remember information about the user across chats,” storing entities, relations, and observations as a JSON graph on the local filesystem under an MIT license [35]. Because HCP already commits to MCP as its own interoperability transport (Section 5.1), this reference server demonstrates that a local, on-device knowledge graph is not merely compatible with HCP by coincidence, but is already shipping, maintained infrastructure inside the exact protocol stack HCP’s reference implementation depends on. OpenJarvis’s “interchangeable memory backends” primitive is a second, independent confirmation that production-grade local memory representations now run at the framework level rather than only in research demos [18]. Neither system implements the expanded paper’s Encapsulation, Control, or Security semantics: the MCP reference memory server has no notion of per-category consent, revocation, or audit – any client with tool access can read or write the entire graph – and OpenJarvis’s memory backends are similarly ungated with respect to scoped sharing. Both are also profile-shaped stores. CIP’s remaining connective layer would be an on-device graph, built from a custody plane rather than from extracted entities, gated by scoped consent, and exposed through the workshop’s Interoperability Interface rather than as an ungated shared graph. SOS is the custody plane in that sentence, not the gated interface.

6.5 Application and Task Assistance: Proposed CIP Work, Not Present SOS Behavior

The third HCP gap is the least resolved of the three, and the prototype does not resolve it. The expanded HCP paper specifies Read, Write, Update, and Delete access to a memory store and audit logging of who accessed what and when [2]; the workshop report names a Policy & Authorization Service and an Audit & Provenance Subsystem [3]. None of these publications demonstrate a downstream agent actually *using* authorized context to execute a real task – booking, drafting, purchasing, scheduling – under the protocol’s consent and audit guarantees. CIP’s proposed agentic layer would close this specific gap by implementing task-oriented sub-agent delegation on top of, not alongside, those authorization and audit primitives: every sub-agent dispatch and third-party tool call would read its authorized context scope from the Policy & Authorization Service, execute via MCP tool calls subject to the specification’s human-in-the-loop approval guidance [29, 30], and write a structured record to the Audit & Provenance Subsystem. This is a deliberately conservative design choice: it treats the workshop’s five-component architecture as the target interface to implement against, rather than proposing a sixth, competing component. It is also not present SOS behavior. SOS does not dispatch tools against the world, and it does not maintain a runtime disclosure ledger.

Workshop Component [3]	Specification status	SOS prototype (this edition)	Proposed CIP runtime
Memory Store	Specified; reference implementation is centralized PostgreSQL [1]	Local reviewed records, evidence, and sources (Section 6.2)	Policy-enforced store over that custody plane

Workshop Component [3]	Specification status	SOS prototype (this edition)	Proposed CIP runtime
Policy & Authorization Service	Specified conceptually; not demonstrated at delegation scale	Not built	Default-deny, purpose-limited, revocable grants (Section 6.6)
Context Router (Memory Manager)	Specified conceptually	Local retrieval for the operator; no third-party router	Minimal provenance-bearing bundles from a grant
Audit & Provenance Subsystem	Specified; store-access audit described in the expanded HCP paper [2]	Structural evidence-route validation; no runtime access ledger	Ledger of reads, grants, proposed writes, dispatches, and outcomes
Interoperability Interface	Specified as add/search/update/delete [3]	Local CLI control plane; no MCP server	Versioned local gateway; MCP as transport, not as the store

6.6 Proposed CIP Runtime Data Flow

CIP would preserve the SOS custody plane and add a mediated runtime in front of it. A request would name an actor, a purpose, and a requested scope at a local gateway. A policy service would issue or refuse a grant. A context router would turn an allowed grant into a minimal, provenance-bearing bundle drawn from the local graph and its evidence. Replaceable models and tools would receive that bundle, or authorized parameters, and would not mount the custody filesystem. Returned results would become either a Tier 2 evidence artifact or a reviewable proposal; they would not become silent memory. Durable Tier 1 mutation would remain an operator-reviewed decision.

Where a remote agent is needed, CIP could hand off via A2A without exposing the local graph’s internal structure [34]. Each read, grant, proposed write, dispatch, and tool call would be recorded in the Audit & Provenance Subsystem. That flow is proposed. The prototype implements only the plane the flow would be allowed to touch.

7 Privacy, Security, and Governance

7.1 Regulatory Implications: GDPR and the Right to Erasure

GDPR Article 17’s right to erasure and Article 20’s right to portability sit in tension with LLM memory. “AI models do not store information in discrete entries,” so “once personal data is integrated into a model’s parameters, removal becomes nearly infeasible without costly retraining or experimental machine unlearning methods” [36]. The European Data Protection Board notes that satisfying the right to erasure “requires reversing the memorisation of personal data by the model,” with options ranging from full retraining to sharded retraining and parameter-level unlearning [37]. This problem largely dissolves for a well-designed memory *store* external to model weights: production vector databases now offer metadata-filtered bulk deletion explicitly marketed to “honor Right to be Forgotten requests (GDPR) with one API or SDK call” [38], which in principle lets an operator tag embeddings with a `data_subject_id` and purge by filter rather than collecting IDs by hand. In practice this guarantee is weaker than it appears: recent work on HNSW-indexed vector stores shows that soft-deleted embeddings typically remain physically present in the on-disk index, and an embedding-inversion attack can reconstruct deleted personal identifiers and structured fields directly from the untouched vectors — 25.5% of person names and 46.4% of locations on its headline dataset, and up to 100% on narrow structured fields such as age and gender — even after

a delete-by-ID or delete-by-metadata call has completed [39]. A local graph store with a literal, relationally-enforced `DELETE CASCADE` would avoid this failure mode by construction: deleting a node removes its properties and every dependent edge in a single transaction at the storage layer, rather than leaving a soft-deleted, potentially recoverable index entry behind – a direct architectural argument for a future local-graph memory store, and the same argument the expanded paper’s Security attribute rests on, made here at the level of implementation mechanics rather than design principle [2]. The SOS prototype’s ordinary-file custody makes deletion inspectable as file removal and archive relocation; it does not, in this edition, claim a database-level cascade. Vector-index residue remains a reason not to treat embeddings as the system of record.

7.2 Threat Model: Memory Poisoning

Persistent memory expands the attack surface. The MINJA (Memory Injection Attack) demonstrates that an adversary, “without assuming that the attacker can directly modify the memory bank,” can inject malicious records “by only interacting with the agent via queries and output observations,” using bridging steps that later cause harmful reasoning when a victim issues a related query [40]. Because “any user [can] influence agent memory,” this is a low-barrier, high-impact attack class [40], and it is not addressed by HCP’s published threat model, which stops at unauthorized preference disclosure rather than adversarial injection into the store itself [1, 2]. A review-gated write path, as in the SOS debrief protocol, narrows the query-only injection surface relative to systems that extract and persist memory automatically. It does not eliminate poisoning: an operator can still be induced to accept a bad claim, and a future CIP router that injects bundles into models reintroduces a smaller, grant-shaped version of the same risk. Automatic memory writers remain the more permissive target.

7.3 Threat Model: MCP Tool Poisoning

Delegated reasoning imports the tool layer’s vulnerabilities. Invariant Labs documents MCP Tool Poisoning as an indirect prompt injection in which “malicious instructions are embedded within MCP tool descriptions that are invisible to users but visible to AI models”; because “AI models see the complete tool descriptions ...while users typically only see simplified versions in their UI,” the model reads attacker instructions the human never sees [41]. Concrete 2025 CVEs illustrate the class: CurXecute (CVE-2025-54135, CVSSv3 8.5), a remote-code-execution flaw in the Cursor editor reachable via prompt injection when an MCP-connected tool is used to rewrite `mcp.json`, and MCPoison (CVE-2025-54136, CVSSv3 7.2), which abuses trust-by-name rather than trust-by-content in approved MCP server configurations to smuggle in silently-executed commands [42]. JFrog’s disclosure of CVE-2025-6515 (“Prompt Hijacking”) showed session-ID reuse could cause a model to “present the user with the attacker’s intended response, while dropping the response for the user’s original request” [43]. Recommended mitigations include version and hash pinning of MCP servers and tools, cross-server dataflow boundaries, and refusing to auto-approve tool calls [41]. This threat class belongs to CIP’s proposed agentic layer (Section 6.5), which would execute third-party MCP tools on the user’s behalf. It is not a present SOS attack surface, because SOS does not presently expose an MCP tool host. It is also not addressed by any of the three HCP publications, whose threat model does not extend past the memory store’s own access controls.

8 Comparative Positioning

The comparison below bounds the design space with six entries. HCP’s own protocol family, not Sovereign OS, specifies interoperability and audit as named requirements. SOS’s present differentiation is confined to a published local custody and provenance plane. CIP’s remaining differentiation is the still-unbuilt combination of a policy boundary, a context router, mediated interoperability, and audited delegation in front of such a plane.

System	Protocol Status	Open-Source?	Local Memory Graph?	Demonstrated Agentic Delegation?
HCP (2025 preprint / Dec 2025 working paper)	Specifies interoperability, encapsulation, control, security [1, 2]	No – public repo, unlicensed, no releases [10]	No – centralized PostgreSQL	Gestured at (MCP/A2A mentioned), not demonstrated
Global Memory Workshop Architecture	Specifies five-component reference architecture [3]	No – report only, no codebase	No – calls for it as future work	No – specification only
HCP-Demo (reference impl.)	Illustrates HCP’s own consent-scoped model; narrative walkthrough, not a deployable backend [10]	No – public repo, no license file, no tagged releases [10]	No – Next.js narrative app, no graph store	No – no delegation layer
OpenJarvis	No relation to HCP; general-purpose local-agent framework [18]	Yes – Apache 2.0, public releases [18]	No HCP-compliant graph – interchangeable, ungated memory backends [18]	Yes – on-device agent + MCP tool use, benchmark-evaluated [18]
SOS prototype (this edition)	Custody and provenance plane; not CIP, not HCP, not MCP [5]	Yes – MIT, public control plane 1.6.0; author-maintained, not community-governed	Working – local typed records and evidence routes, not a gated graph API	No – review-gated authoring, not delegated tool execution
CIP (proposed)	Edge profile for mediated access to a custody plane; layered over HCP and MCP	Planned – permissive license and public codebase	Planned gated interface over a plane like SOS	Planned – sub-agent and MCP tool execution under protocol audit

HCP-Demo demonstrates the consent-scoped preference model as narrative, not as a licensed, deployable backend, and implements neither a local memory graph nor agentic task delegation [10]. OpenJarvis demonstrates that a fully local, open-source agentic stack with interchangeable memory backends and MCP tool adapters can already match near-frontier cloud accuracy on real benchmarks at a fraction of the cost and latency [18] – but it implements no consent, revocation, audit, or scoped-sharing semantics over that memory; any tool with access reads the whole store. Neither is a competitor to the SOS prototype’s narrower custody claim, and neither is CIP. HCP-Demo has the protocol’s sharing semantics but not local custody of source, observation, and accepted claim. OpenJarvis has local agentic execution but not those semantics and not a review-gated custody plane. The unclaimed remainder – not local AI in general, and not agentic delegation in general – is a mediation layer that can disclose provenance-bearing context without turning the store into an ungated profile. That remainder is CIP. It is not a claim that SOS has already built the HCP-MCP stack.

9 Challenges and Future Directions

Several open problems remain within this scope. **Memory consistency and forgetting** are unresolved in general: bi-temporal graphs with edge invalidation [15] handle changing facts more gracefully than flat RAG, but no consensus exists on when an agent should proactively forget, and reflection weights in memory-stream architectures remain hand-tuned rather than learned [14]. **Benchmark reliability** is a standing concern: the wide divergence between vendor-published memory-system accuracy figures, including a competing vendor’s own unfavorable re-measurement of its rivals [17], means any later evaluation of SOS or CIP against LOCOMO, Deep Memory Retrieval, or LongMemEval must be run by a disinterested party rather than cited from any vendor’s claims, and none of these benchmarks yet measure delegation-audit completeness, evidence-route inspectability, or poisoning resistance specifically [40, 44]. **Cross-device and cross-vendor interoperability** remains weak industry-wide despite HCP’s specification work: no fetched vendor source documents a full, standards-based export of its memory store, and portability today is largely limited to Claude’s import of other chatbots’ exports [25]. **On-device model capability** is the workshop’s own identified constraint: distillation, hybrid small-model/deterministic pipelines, and tiered execution are called for but not yet demonstrated at the fidelity cloud-hosted models achieve [3]. The SOS prototype does not inherit a local-inference requirement; CIP’s proposed on-device mediation layer would.

On institutional trajectory, the most consequential recent development is the speed at which the HCP research program has moved from a 2025 multi-author preprint spanning MIT, Persona Labs, Oxford, OpenMined, and Microsoft Research to a December 2025 Stanford working paper and a March 2026 workshop report with a named five-component architecture [1, 2, 3]. This is strong evidence that the protocol layer this paper’s founding sovereignty argument depends on is no longer a speculative proposal but an active, multi-institution standardization effort, alongside MCP’s continued adoption across OpenAI [31] and Google DeepMind [32] and A2A’s transfer to the Linux Foundation [45]. The Sovereign OS Project’s role in that trajectory is not to originate those protocols. It is to keep the stored object from collapsing into a preference profile, to show that a local custody plane can be published as ordinary files, and to specify CIP as the still-unbuilt gate in front of that plane.

10 Conclusion

Personal AI memory is the newest round of a decades-old cycle between centralized and personally controlled computing. The Human Context Protocol research program – a 2025 preprint, a December 2025 expanded working paper with named store actions and audit language, and a March 2026 workshop report decomposing a user-governed context layer into a five-component architecture – has moved the reclamation argument from philosophy to engineering with unusual speed. This paper contributes the Context Inversion Protocol as an edge-first implementation profile for mediated access to persistent human context, layered over that existing stack rather than offered as a second Human Context Protocol.

Version 0.2.0 changes the research claim. This edition reports the SOS control plane 1.6.0 as a published local custody and provenance prototype: captured reality, deterministic observation, and accepted knowledge remain distinct, inspectable, and revisable [5]. That is an author-reported existence claim, not an independently verified result. It does not claim the full policy, context-routing, delegated-action, and audit runtime envisioned by CIP. That distinction is the paper’s

credibility condition. HCP remains the protocol-level language this paper uses for scoped disclosure; MCP remains the transport this paper uses for a future gateway; SOS is a published control plane showing that durable context need not become a vendor-owned, or even user-owned, personality profile that silently decides what an AI knows. Independent verification of that prototype remains future work, consistent with the standard applied to other sovereignty claims in Section 4.2.

References

- [1] Shah, A. V., South, T., Evans, T., Kirk, H. R., Trask, A., Weyl, E. G., & Bakker, M. A. (2025). Robust AI Personalization via The Human Context Protocol. Preprint <https://hcp.loyalagents.org/hcp-paper.pdf>
- [2] Shah, A. V., South, T., Evans, T., Kirk, H. R., Pei, J., Trask, A., Weyl, E. G., & Bakker, M. A. (2025, December 18). Robust AI Personalization Controls: The Human Context Protocol. Stanford Digital Economy Lab working paper <https://digitaleconomy.stanford.edu/publication/robust-ai-personalization-controls-the-human-context-protocol/>
- [3] Global Memory Workshop. (2026, March). The Future of Personal AI: Portable and Persistent Personal Memory through a Unified Human Context Protocol. Final report, March 11–12, 2026. Stanford Digital Economy Lab <https://digitaleconomy.stanford.edu/app/uploads/2026/06/GlobalMemoryWorkshop.pdf>
- [4] Anthropic (2024). Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>
- [5] Potter, R. E. (2026). Sovereign OS Control Plane (Version 1.6.0) [Computer software]. MIT License <https://github.com/robertpotter-dev/sos-control-plane>
- [6] BBC News (2024). Tim Berners-Lee: I invented the web. Here’s how we take it back. <https://www.bbc.com/news/business-68286395>
- [7] Solid Project. About Solid. <https://solidproject.org/about>
- [8] Inrupt. Key Concepts. <https://docs.inrupt.com/getting-started/key-concepts>
- [9] Schneier, B. (2020). Inrupt: Tim Berners-Lee’s Solid. Schneier on Security. https://www.schneier.com/blog/archives/2020/02/inrupt_tim_bern.html
- [10] South, T., et al. HCP Demo. GitHub <https://github.com/tobinsouth/hcp-demo>
- [11] Packer, C., et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. <https://arxiv.org/abs/2310.08560>
- [12] Chhikara, P., et al. (2025). Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413. <https://arxiv.org/abs/2504.19413>
- [13] Anthropic (2025). Effective Context Engineering for AI Agents. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- [14] Park, J. S., et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442. <https://arxiv.org/abs/2304.03442>
- [15] Rasmussen, P., et al. (2025). Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956. <https://arxiv.org/abs/2501.13956>
- [16] Zep. Research Benchmarks. <https://www.getzep.com/research/>
- [17] Vectorize (2026). Mem0 vs Zep: AI Agent Memory Compared. <https://vectorize.io/articles/mem0-vs-zep>
- [18] Saad-Falcon, J., Narayan, A., Manihani, R., Bhathal, T., Shandilya, H., Akengin, H. O., Bo, G., Park, A., Hart, M., Costello, C., Li, C., Ré, C., & Mirhoseini, A. (2026). OpenJarvis: Personal AI, On Personal Devices. Stanford Hazy Research / Scaling Intelligence Lab. arXiv:2605.17172. <https://arxiv.org/abs/2605.17172>
- [19] Apple (2024). Platforms State of the Union – WWDC24. <https://developer.apple.com/videos/play/wwdc2024/102/>

- [20] Apple Support. Apple Intelligence and Privacy. <https://support.apple.com/guide/iphone/apple-intelligence-and-privacy-iphe3f499e0e/ios>
- [21] Dittmar, Y., Stephan, M. J., Völkl, T., Hollick, M., & Classen, J. (2026). Unlocking Apple’s Private Cloud Compute: An Analysis of Privacy-Preserving Artificial Intelligence. arXiv:2605.24239. <https://arxiv.org/abs/2605.24239>
- [22] Khemani, S. How Gemini’s Memory Works. <https://www.shloked.com/writing/gemini-memory>
- [23] OpenAI Help Center. How do I view or delete a memory? <https://help.openai.com/en/articles/8983145-how-do-i-view-or-delete-a-memory>
- [24] Anthropic. Use Claude’s chat search and memory to build on previous context. <https://support.anthropic.com/en/articles/11817273-using-claude-s-chat-search-and-memory-to-build-on-previous-context>
- [25] Anthropic. Import and export your memory from Claude. <https://support.anthropic.com/en/articles/12123587-import-and-export-your-memory-from-claude>
- [26] Google. Temporary Chats and Privacy Controls in Gemini. <https://blog.google/products-and-platforms/products/gemini/temporary-chats-privacy-controls/>
- [27] Microsoft Support. Retrace Your Steps with Recall. <https://support.microsoft.com/en-us/windows/ai/ai-features/retrace-your-steps-with-recall>
- [28] Perplexity Help Center. Memory. <https://www.perplexity.ai/help-center/en/articles/10968016-memory>
- [29] Model Context Protocol. (2025–2026). Specification <https://modelcontextprotocol.io/specification/>
- [30] SAP Insiders. Model Context Protocol (MCP): A Deep Dive. <https://sapinsiders.org/posts/model-context-protocol-mcp-deep-dive>
- [31] TechCrunch (2025). OpenAI adopts rival Anthropic’s standard for connecting AI models to data. <https://techcrunch.com/2025/03/26/openai-adopts-rival-anthropics-standard-for-connecting-ai-models-to-data/>
- [32] Google (2025). Gemini 2.5: Our most intelligent models are getting even better. <https://blog.google/innovation-and-ai/models-and-research/google-deepmind/google-gemini-updates-io-2025/>
- [33] Google Developers Blog (2025). A2A: A New Era of Agent Interoperability. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>
- [34] A2A Project. Agent2Agent Protocol. GitHub. <https://github.com/a2aproject/A2A>
- [35] Model Context Protocol Contributors. Knowledge Graph Memory Server. GitHub <https://github.com/modelcontextprotocol/servers/tree/main/src/memory>
- [36] Tech Policy Press. The Right to Be Forgotten Is Dead: Data Lives Forever in AI. <https://techpolicy.press/the-right-to-be-forgotten-is-dead-data-lives-forever-in-ai>
- [37] European Data Protection Board (2025). AI: Effective Implementation of Data Subjects’ Rights. https://www.edpb.europa.eu/system/files/2025-01/d2-ai-effective-implementation-of-data-subjects-rights_en.pdf
- [38] Wang-Tomic, L., Johnson, G. (2025). New Bulk Data Operations: Update, Delete, and Fetch by Metadata. Pinecone. <https://www.pinecone.io/blog/update-delete-and-fetch-by-metadata/>
- [39] Chakrabortii, C., Alvarado, J.G., Abdulofizova, S., Dwivedi, S. (2026). Ghost Vectors: Soft-Deleted Embeddings Remain Reconstructible in HNSW Vector Databases. arXiv:2606.18497. <https://arxiv.org/abs/2606.18497>
- [40] Dong, Y., et al. (2025). Memory Injection Attacks on LLM Agents via Query-Only Interaction. arXiv:2503.03704. <https://arxiv.org/abs/2503.03704>
- [41] Invariant Labs. MCP Security Notification: Tool Poisoning Attacks. <https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks>

- [42] Narang, S. (2025). FAQ: CVE-2025-54135 and CVE-2025-54136 Vulnerabilities in Cursor (CurXecute and MCPoison). Tenable. <https://www.tenable.com/blog/faq-cve-2025-54135-cve-2025-54136-vulnerabilities-in-cursor-curxecute-mcpoison>
- [43] JFrog (2025). MCP Prompt Hijacking Vulnerability. <https://jfrog.com/blog/mcp-prompt-hijacking-vulnerability/>
- [44] OWASP. MCP Tool Poisoning. https://owasp.org/www-community/attacks/MCP_Tool_Poisoning
- [45] Google Cloud Blog. Agent2Agent Protocol Is Getting an Upgrade. <https://cloud.google.com/blog/products/ai-machine-learning/agent2agent-protocol-is-getting-an-upgrade>